

1. 设有 SDD 如下:

产生式	语义规则
$S \rightarrow B\#$	$S.val = B.val$
$B \rightarrow B0$	$B.val = 2 \times B_1.val$
$B \rightarrow B1$	$B.val = 2 \times B_1.val + 1$
$B \rightarrow 1$	$B.val = 1$

给出 101101 的注释语法分析树。

2. 含小数点的二进制数文法定义如下:

$S \rightarrow L.L|L$

$L \rightarrow LB|B$

$B \rightarrow 0|1$

(1) 根据上述文法设计一个 SDD, 实现二进制数到十进制数的翻译, 比如 101.101 翻译为 5.625。

(2) 给出 101.101 翻译为 5.625 的注释语法分析树。

3. (选做) 在保证语义相同的情况下, 改写 1 题中的 SDD, 使文法不是左递归的。

4. 设包含有数组引用的复制表达式的 SDD 如下:

产生式	语义规则
$S \rightarrow i=E;$	$gen(= E.place - i)$
$S \rightarrow L=E;$	$gen([] = E.place - L.base[L.addr])$
$E \rightarrow E+E$	$E.place = newtemp(); gen(+ E_1.place E_1.place E.place)$
$E \rightarrow i$	$E.place = i$
$E \rightarrow L$	$E.place = newtemp(); gen(= [] L.base[L.addr] - E.place)$
$L \rightarrow i[E]$	$L.base = i; L.addr = newtemp(); gen(* E.place L.elem.width L.addr)$
$L \rightarrow L[E]$	$L.base = L_1.base; L.type = L_1.type; t = newtemp(); L.addr = newtemp(); gen(* E.place L.elem.width t);$ $gen(+ L_1.addr t L.addr)$

给出下列赋值语句的四元式代码:

(1) $x = a[i][j] + b[i][j]$

(2) $a[b[i][j]][c[k]] = a[i][j]$

说明:

a) $gen()$ 是产生四元式代码的函数, 参数依次为操作符、第一操作数、第二操作数、计算结果;

b) $newtemp()$ 是临时变量申请函数;

c) 关于数组中用到的属性 width 的 SDD 如下:

产生式	语义规则
$D \rightarrow B i C$	$w = B.width; i.width = C.width$
$B \rightarrow int$	$B.width = 4$
$B \rightarrow float$	$B.width = 8$
$C \rightarrow \epsilon$	$C.width = w$
$C \rightarrow [num]C$	$C.width = num.value \times C_1.width; C.elem.width = C_1.width$

d) 语句中的数组说明: $int\ a[10][20], b[10][20]; c[10]$ 。

5. 设有布尔表达式以及条件语句的 SDD 如下:

产生式	语义规则
-----	------

B→E	B.true=makelist(nextinstr); B.false=makelist(nextinstr+1); gen(J _T E.PLACE - 0); gen(J _F E.PLACE - 0)
B→E=E	B.PLACE=NEWTEMP; gen(= E1.PLACE E2.PLACE B.PLACE); B.true=makelist(nextinstr) B.false=makelist(nextinstr+1); gen(J _T B.PLACE - 0); gen(J _F B.PLACE - 0)
B→!B	B.true=B ₁ .false; B.false=B ₁ .true
B→(B)	B.true=B ₁ .true; B.false=B ₁ .false
B→B MB	backpatch(B ₁ .false,M.instr); B.true=merge(B ₁ .true,B ₂ .true); B.false=B ₂ .false
B→B&&MB	backpatch(B ₁ .true,M.instr); B.true=B ₂ .true; B.false=merge(B ₁ .false,B ₂ .false)
S→if(B)MSNelseMS	backpatch(B.true,M1.instr); backpatch(B.false,M2.instr); S.next=merge(S ₁ .next,N.next,S ₂ .next)
S→whileM (B) MS	backpatch(S ₁ .next,M1.instr); backpatch(B.true,M2.instr); S.next=B.false;gen(J - - M ₁ .instr)
M→ε	M.instr=nextinstr
N→ε	N.next=makelist(nextinstr); gen(J - -0)
S→{L}	S.next=L.next
S→i=E;	S.next=null
L→LMS	{backpatch(L ₁ .next, M.instr); L.next=S.next;}
L→S	L.next=S.next

SDD 的说明:

a) 四元式编号作为跳转地址使用

b) 函数 makelist(x)表示出现了标号的引用时不知道地址的情况，函数参数 x 为标号的使用位置，单遍处理生成中间代码需要使用拉链—返填技术，假设通过标号表记录相关信息，此处为标号的引用出现，按标号引用记录入标号表。

c)函数 backpatch(x,y)表示标号 x 的定义位置在 y,需要返填标号 x 的使用位置。

d)函数 merge(x,y,z,...)表示标号 x,y,z...合并为一个标号

e)nextinstr 表示下一条四元式的编号

有程序段:

```
if (a==b&&(c==d || e==f))
{while(!(x==y))
    x=x+y;
  s=x;}
else
```

```
  s=y;
z=s;
```

问题:

(1) 给出该程序段的语法分析树(表达式 E 结构使用 4 题的)。

(2) 给出该程序段语义处理后的四元式形式的目标代码(代码中的表达式 E 结构使用 4 题的 SDD)。

(3) 给出该程序段语义处理后的拉链-返填使用的标号表。

标号表按以下示例内容填写:

标号名	定义否 (1/0)	返填顺序
L _i	1	⑤→②→①
...		